

Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)»

**«Исследование методов определения характеристик транспортного средства
по видеоизображениям»**

(бакалаврская работа)

Студент:

Клименко Виталий Евгеньевич

Научный руководитель:

Назаров Алексей Николаевич

Москва
2026

Аннотация

Ключевые слова: детекция транспортных средств, распознавание номерных знаков, классификация цвета автомобиля, тип кузова, YOLOv8, ResNet18, EasyOCR, transfer learning, видеоаналитика, глубокое обучение.

Выпускная квалификационная работа посвящена исследованию и разработке программного комплекса для автоматического определения характеристик транспортных средств по видеопотоку с камер наблюдения: детекции автомобиля в кадре, локализации номерного знака, распознавания символов, а также классификации цвета и типа кузова.

Предлагается конвейерная архитектура на основе методов глубокого обучения: YOLOv8n — для детекции ТС и номерных знаков, ResNet18 — для классификации цвета (9 классов) и типа кузова (6 классов), EasyOCR — для оптического распознавания символов кириллицы.

Обучение и тестирование проводились на открытых датасетах (VCoR, CompCars, Kaggle car-plate-detection). Детектор номерных знаков YOLOv8n достиг $mAP@0.50 = 0.889$. Классификатор цвета ResNet18 достиг Accuracy top-1 = 0.935. Классификатор типа кузова ResNet18 — Accuracy top-1 = 0.784. EasyOCR обеспечивает CRR = 0.975 на синтетических номерах. Проведено сравнительное исследование ResNet18 и MobileNetV2. Результатом является работоспособный прототип системы видеоаналитики транспорта.

Contents

Аннотация	1
1 Введение	4
1.1 Актуальность задачи	4
1.2 Постановка задачи	4
1.3 Обзор методов детекции транспортных средств	4
1.3.1 Сравнение подходов к детекции	4
1.3.2 Двухэтапные детекторы (Faster R-CNN)	5
1.3.3 Одноэтапные детекторы (YOLO)	5
1.4 Обзор методов автоматического распознавания номерных знаков (ANPR)	5
1.4.1 Сравнение ANPR-подходов	5
1.5 Обзор методов распознавания марки и типа кузова (VMMR)	5
1.6 Обзор методов определения цвета автомобиля	6
1.7 Цель и задачи работы	6
2 Описание выбранных методов и датасетов	8
2.1 Архитектура системы	8
2.2 Теоретические основы глубокого обучения	8
2.2.1 Сверточные нейронные сети	8
2.2.2 Функции активации	9
2.2.3 Pooling и агрегация признаков	9
2.2.4 Пакетная нормализация	9
2.3 Детекция транспортных средств: YOLOv8	10
2.3.1 Внутренняя архитектура YOLOv8	10
2.3.2 Эволюция семейства YOLO	11
2.3.3 Backbone: CSPDarkNet с блоками C2f	11
2.3.4 Neck: Path Aggregation Network	12
2.3.5 Anchor-free голова и Distribution Focal Loss	12
2.3.6 Параметры инференса	13
2.4 Детекция номерных знаков: дообученный YOLOv8n	13
2.4.1 Схема transfer learning	13
2.4.2 Параметры обучения	13
2.5 Оптическое распознавание номеров: EasyOCR	13
2.6 Классификация цвета и типа кузова: ResNet18	14
2.6.1 Архитектура ResNet18	14
2.6.2 Двухфазное дообучение	15
2.6.3 Проблема затухающего градиента в глубоких сетях	16
2.6.4 Теоретическое обоснование residual connections	16
2.6.5 ImageNet-предобучение и стратегии transfer learning	16
2.6.6 Гиперпараметры обучения	17
2.6.7 Аугментация данных	17
2.7 Алгоритмы оптимизации	18
2.7.1 Стохастический градиентный спуск с импульсом	18
2.7.2 Adam	18
2.7.3 Планировщики скорости обучения	18

2.8	Методы регуляризации	19
2.8.1	L2-регуляризация (weight decay)	19
2.8.2	Ранняя остановка	19
2.8.3	Аугментация как регуляризация	19
2.9	Сравниваемая архитектура: MobileNetV2	20
2.10	Датасеты	20
2.10.1	VCoR — классификация цвета	21
2.10.2	CompCars — классификация типа кузова	21
3	Описание экспериментов	22
3.1	Условия проведения экспериментов	22
3.2	Эксперимент 1: дообучение детектора номерных знаков	22
3.2.1	Протокол и обоснование гиперпараметров	22
3.2.2	Результаты	23
3.3	Эксперимент 2: классификатор цвета	23
3.3.1	Протокол и обоснование гиперпараметров	23
3.3.2	Результаты	24
3.4	Эксперимент 3: классификатор типа кузова	24
3.4.1	Протокол и обоснование гиперпараметров	24
3.4.2	Результаты	25
3.5	Эксперимент 4: сравнение архитектур ResNet18 и MobileNetV2	26
3.5.1	Постановка и обоснование методологии	26
3.5.2	Результаты	26
3.6	Эксперимент 5: финальный конвейер	26
3.6.1	Тест на одиночном изображении	26
3.6.2	Метрики производительности	27
3.6.3	Итоговая таблица результатов	27
3.6.4	Анализ узких мест и практические рекомендации	27
4	Выводы	28

1 Введение

1.1 Актуальность задачи

В настоящее время системы видеонаблюдения широко применяются для мониторинга дорожного движения, обеспечения безопасности и контроля доступа на объекты. Объём видеоданных, поступающих с камер, постоянно растёт, что делает ручную обработку информации неэффективной и трудоёмкой [3].

Практический интерес представляет автоматическое определение характеристик автомобиля: государственного регистрационного знака, марки, модели и цвета. Эти данные применяются в задачах контроля проезда, поиска транспортных средств и учёта трафика [4]. Однако качество видеозаписей с камер наблюдения часто ограничено низким разрешением, изменениями освещённости, погодными условиями и движением объектов.

1.2 Постановка задачи

Задача анализа транспортных средств по видеоданным представляется как последовательность следующих этапов:

1. детекция автомобиля в кадре;
2. локализация номерного знака внутри области автомобиля;
3. оптическое распознавание символов номера;
4. классификация визуальных атрибутов: цвет и тип кузова.

Формально результат детекции объектов на изображении:

$$D = \{(b_i, c_i, p_i)\}_{i=1}^N, \quad (1.1)$$

где $b_i = (x_i, y_i, w_i, h_i)$ — ограничивающий прямоугольник, c_i — класс объекта, p_i — уверенность детекции.

Качество локализации оценивается через Intersection over Union:

$$\text{IoU} = \frac{S(B_{\text{pred}} \cap B_{\text{gt}})}{S(B_{\text{pred}} \cup B_{\text{gt}})}. \quad (1.2)$$

1.3 Обзор методов детекции транспортных средств

1.3.1 Сравнение подходов к детекции

Эволюция методов детекции объектов прошла несколько этапов. Таблица 1.1 суммирует основные подходы.

Table 1.1: Сравнение методов детекции объектов

Метод	Принцип	Достоинства	mAP	FPS
HOG+SVM	Ручные признаки, скользящее окно	Прост в реализации, нет GPU	низкий	5–15
Наар-каскады	Каскад слабых классификаторов	Очень быстрые на CPU	низкий	30+
Faster R-CNN	Двухэтапная сеть, RPN+cls	Высокая точность	высокий	5–15
SSD	Одноэтапная, anchor-based	Баланс скорости и точности	средний	46+
YOLOv8n	Одноэтапная, anchor-free	Скорость + точность	высокий	100+

1.3.2 Двухэтапные детекторы (Faster R-CNN)

Detectors семейства Faster R-CNN работают в два этапа: Region Proposal Network генерирует кандидаты, затем классификационная голова уточняет координаты. Подход обеспечивает высокую точность, но требователен к вычислительным ресурсам и не обеспечивает скорость, достаточную для обработки видео в реальном времени.

1.3.3 Одноэтапные детекторы (YOLO)

YOLOv8 [4] решает задачу детекции за один проход по сети. Anchor-free архитектура, CSPDarkNet backbone и FPN neck обеспечивают работу на GPU со скоростью 100+ FPS при конкурентоспособной точности.

Обоснование выбора: YOLOv8n оптимален для видеопотока в реальном времени и совместим с форматами ONNX/TensorRT.

1.4 Обзор методов автоматического распознавания номерных знаков (ANPR)

Типовая ANPR-система включает несколько последовательных этапов, схема которых представлена на рисунке 1.1.

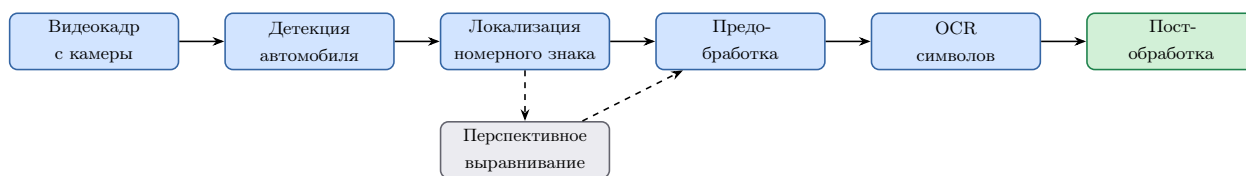


Рисунок 1.1: Типовая схема ANPR-системы

1.4.1 Сравнение ANPR-подходов

Table 1.2: Сравнение подходов к ANPR

Подход	Описание	mAP plate	Усл. освещения
Морфология + шаблоны	Пороговая обработка + шаблонный OCR	< 0.70	Плохо
HOG + SVM + Tesseract	HOG-дескриптор для локализации + Tesseract OCR	0.75–0.80	Удовлетворительно
YOLO + CRNN	YOLO-детекция + seq2seq OCR	0.88–0.93	Хорошо
YOLOv8 + EasyOCR	Anchor-free детекция + CTC OCR	>0.90	Хорошо

Обоснование выбора: YOLOv8n (fine-tuned) + EasyOCR обеспечивает готовую поддержку кириллицы без обучения собственной OCR-модели.

1.5 Обзор методов распознавания марки и типа кузова (VMMR)

Распознавание марки/модели автомобиля (VMMR) относится к задачам тонкой многоклассовой классификации [5, 6]. Задача:

$$\hat{y} = \arg \max_k P(y = k | x), \quad (1.3)$$

где x — изображение, k — класс марки или типа кузова.

Table 1.3: Сравнение архитектур классификаторов для VMNR

Архитектура	Особенности	Params, М	Top-1 Acc	FPS
AlexNet	Первая глубокая CNN для ImageNet	61	0.74–0.80	высокий
VGG16	Глубокая однородная архитектура	138	0.85–0.89	низкий
ResNet18	Residual connections, лёгкая	11.7	0.88–0.92	высокий
MobileNetV2	Depthwise sep. conv., инв. бутылочное горлышко	3.4	0.86–0.91	очень высокий
EfficientNet-B0	Составной масштаб (глубина/ширина/разрешение)	5.3	0.90–0.94	средний

Обоснование выбора ResNet18: оптимальное соотношение точности и скорости; 11.7М параметров позволяют дообучить модель на ограниченных данных без переобучения.

1.6 Обзор методов определения цвета автомобиля

Table 1.4: Сравнение методов классификации цвета автомобиля

Метод	Принцип	Ассурасу	Устойчивость к освещению
k-means (RGB)	Кластеризация пикселей	0.70–0.81	Низкая
k-means (HSV)	Кластеризация в HSV-пространстве	0.78–0.85	Средняя
SVM + цветовые гистограммы	Гистограммы + SVM	0.83–0.88	Средняя
CNN (ResNet18)	Сквозное обучение	>0.90	Высокая

Для повышения устойчивости в видеопотоке применяется агрегация предсказаний по нескольким кадрам [8]:

$$\hat{y} = \arg \max_k \sum_{t=1}^T \mathbf{1}[y_t = k]. \quad (1.4)$$

Обоснование выбора: CNN (ResNet18) неявно учится инвариантности к освещению и превосходит кластеризационные методы на 10–15%.

1.7 Цель и задачи работы

Цель — разработка системы автоматического определения цвета, типа кузова и номерного знака автомобиля по видеопотоку.

Задачи:

1. Провести анализ существующих методов детекции ТС, ANPR, VMNR и определения цвета.
2. Выбрать и обосновать архитектуры моделей для каждой подзадачи.
3. Подготовить обучающие датасеты.

4. Обучить и оценить модели детекции номеров, классификаторы цвета и типа кузова.
5. Интегрировать модели в единый конвейер обработки видео.
6. Провести сравнительный эксперимент ResNet18 vs MobileNetV2.

2 Описание выбранных методов и датасетов

2.1 Архитектура системы

Разработанная система представляет собой последовательный конвейер, обрабатывающий каждый кадр видеопотока. Общая схема приведена на рисунке 2.1.

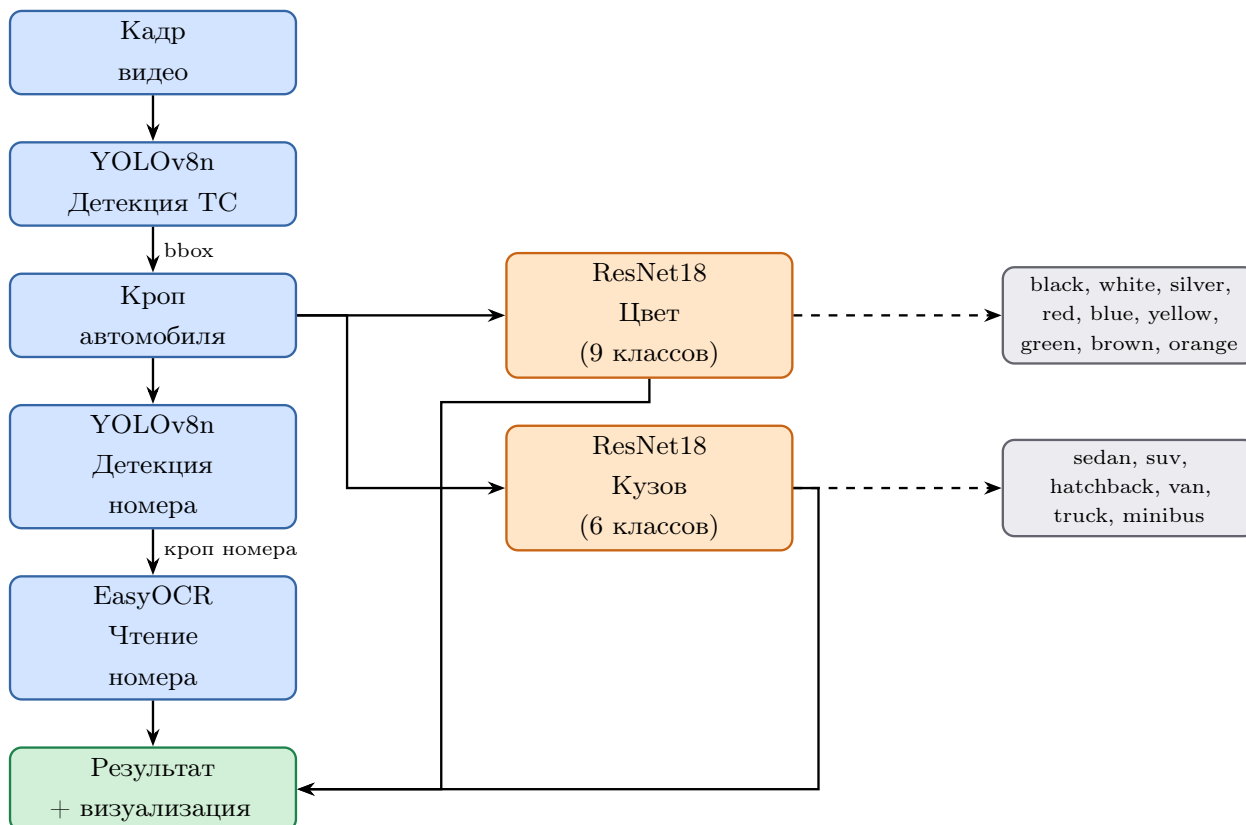


Рисунок 2.1: Схема конвейера обработки видеокadra

2.2 Теоретические основы глубокого обучения

Применяемые в работе модели относятся к классу глубоких сверточных нейронных сетей (Deep Convolutional Neural Networks, DCNN). Данный раздел описывает ключевые строительные блоки, необходимые для понимания архитектур YOLOv8 и ResNet18.

2.2.1 Сверточные нейронные сети

CNN — специализированный класс нейронных сетей для обработки данных с регулярной сеточной топологией. Три ключевых принципа CNN:

- **Разделение весов:** одно ядро применяется ко всем позициям входа, что резко сокращает число параметров по сравнению с полносвязным слоем;
- **Локальная связность:** каждый нейрон реагирует только на рецептивное поле (receptive field) ограниченного размера, что обеспечивает инвариантность к трансляции;
- **Иерархия признаков:** ранние слои кодируют локальные паттерны (края, текстуры), глубокие — семантические концепции (силуэт кузова, форма номерного знака).

Для одноканального входа $I \in \mathbb{R}^{H \times W}$ и ядра $K \in \mathbb{R}^{m \times n}$ дискретная операция кросс-корреляции (в практике машинного обучения называемая «свёрткой»):

$$(I \star K)[i, j] = \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I[i+u, j+v] \cdot K[u, v]. \quad (2.1)$$

В многоканальном случае (C_{in} входных каналов, C_{out} выходных) выходная карта признаков:

$$y_k[i, j] = \sum_{c=1}^{C_{\text{in}}} (I_c \star K_{k,c})[i, j] + b_k, \quad k = 1, \dots, C_{\text{out}}, \quad (2.2)$$

где $K_{k,c}$ — ядро k -го фильтра для входного канала c , b_k — смещение (bias).

Число параметров свёрточного слоя: $m \cdot n \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$, что не зависит от пространственного размера входа $H \times W$. Для сравнения, полносвязный слой потребовал бы $H \cdot W \cdot C_{\text{in}}$ параметров на каждый выходной нейрон.

2.2.2 Функции активации

Для введения нелинейности после каждого свёрточного слоя применяется функция активации. В ResNet18 используется ReLU:

$$\text{ReLU}(x) = \max(0, x). \quad (2.3)$$

ReLU не насыщается на положительной полуоси, градиент равен единице для $x > 0$ — это ключевое свойство, предотвращающее затухание градиентов при прямолинейном прохождении.

В YOLOv8 применяется SiLU (Sigmoid Linear Unit, также известная как Swish):

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}, \quad (2.4)$$

обеспечивающая плавный градиент вблизи нуля и улучшающая сходимость глубоких детекционных сетей.

2.2.3 Pooling и агрегация признаков

Max-pooling с окном $k \times k$ и шагом s уменьшает пространственное разрешение, сохраняя наиболее значимые активации:

$$y[i, j] = \max_{\substack{u \in [0, k) \\ v \in [0, k)}} x[si + u, sj + v]. \quad (2.5)$$

Global Average Pooling (GAP) применяется в конце классификационных сетей перед FC-слоем:

$$\text{GAP}_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W x_c[i, j]. \quad (2.6)$$

GAP сводит пространственную карту к одному числу на канал, что обеспечивает инвариантность к размеру входного изображения и значительно снижает число параметров по сравнению с flatten + FC. В ResNet18 GAP даёт вектор $\mathbb{R}^{512}$ из feature map $7 \times 7 \times 512$.

2.2.4 Пакетная нормализация

Пакетная нормализация (Batch Normalization, BN) нормирует активации по мини-батчу:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta, \quad (2.7)$$

где μ_B и σ_B^2 — выборочные среднее и дисперсия по батчу, γ и β — обучаемые параметры масштабирования и сдвига, $\varepsilon \approx 10^{-5}$ — стабилизирующая константа.

Основные эффекты BN:

1. снижение чувствительности к выбору начальной LR;
2. неявная регуляризация — уменьшает необходимость в Dropout;
3. ускорение сходимости в 10–14 раз по сравнению с сетями без BN при одинаковом расписании LR.

BN присутствует в каждом convolution-блоке как ResNet18, так и YOLOv8, что обусловило выбор относительно высоких начальных LR в обоих случаях.

2.3 Детекция транспортных средств: YOLOv8

2.3.1 Внутренняя архитектура YOLOv8

YOLOv8 состоит из трёх функциональных блоков, показанных на рисунке 2.2.



Рисунок 2.2: Упрощённая схема архитектуры YOLOv8

Функция потерь YOLOv8 объединяет три компонента:

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{dff}} \mathcal{L}_{\text{dff}}, \quad (2.8)$$

где коэффициенты по умолчанию: $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{cls}} = 0.5$, $\lambda_{\text{dff}} = 1.5$.

$\mathcal{L}_{\text{box}}$ — **Complete IoU (CIoU) loss** учитывает одновременно три геометрических аспекта совпадения боксов:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{\text{gt}})}{c^2} + \alpha v, \quad (2.9)$$

где $\rho^2(b, b^{\text{gt}})$ — квадрат евклидова расстояния между центрами предсказанного и эталонного боксов; c — диагональ минимального объёмлющего прямоугольника;

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan \frac{w}{h} \right)^2 \quad (2.10)$$

— штраф за разницу соотношений сторон; $\alpha = v / (1 - \text{IoU} + v)$ — балансирующий коэффициент. В отличие от простого IoU-loss, CIoU обеспечивает корректный ненулевой градиент при непересекающихся боксах и сходится быстрее.

$\mathcal{L}_{\text{cls}}$ — **Binary Cross-Entropy (BCE)** применяется независимо к каждому классу через сигмоид-активацию, что позволяет детектору предсказывать несколько классов одновременно (в отличие от softmax):

$$\mathcal{L}_{\text{cls}} = - \sum_{k=1}^K [y_k \log \hat{p}_k + (1 - y_k) \log(1 - \hat{p}_k)], \quad (2.11)$$

где $\hat{p}_k = \sigma(z_k)$ — предсказанная вероятность класса k .

$\mathcal{L}_{\text{dff}}$ — **Distribution Focal Loss (DFL)** вместо скалярного предсказания координат обучает модель предсказывать дискретное распределение вероятностей по бинам — подробнее см. подраздел 2.3.5.

2.3.2 Эволюция семейства YOLO

Семейство YOLO прошло путь от первой версии (2016) до YOLOv8 (2023), последовательно устраняя ключевые ограничения предыдущих поколений.

Table 2.1: Эволюция ключевых характеристик архитектур YOLO

Версия	Год	Ключевые нововведения	mAP@0.5
YOLOv1	2016	Сетка $S \times S$; B боксов на ячейку; softmax классов	63.4
YOLOv2	2017	Anchor-boxes, BatchNorm, многомасштабное обучение	78.6
YOLOv3	2018	Darknet-53; детекция на 3 масштабах; sigmoid классификаторы	82.1
YOLOv4	2020	CSPDarknet53, PANet, CIoU loss, Mosaic-аугментация	84.0
YOLOv5	2020	AutoAnchor, Focus-слой, адаптивный batching (PyTorch)	84.7
YOLOv6	2022	RepVGG backbone, знаниевая дистилляция	85.0
YOLOv7	2022	Extended-ELAN, составное масштабирование	85.6
YOLOv8	2023	Anchor-free, C2f блоки, decoupled head, DFL loss	86.3

Принципиальное отличие YOLOv8 от предшественников — отказ от anchor-boxes в пользу **anchor-free** подхода. В классических anchor-based детекторах (YOLOv2–v5) необходимо предварительно кластеризовать размеры объектов в датасете для выбора наборов якорей. YOLOv8 вместо этого напрямую предсказывает геометрию бокса, что упрощает адаптацию к нестандартным доменам (например, номерные знаки имеют специфическое соотношение сторон $\approx 4:1$, плохо покрываемое универсальными COCO-якорями).

2.3.3 Backbone: CSPDarkNet с блоками C2f

Backbone YOLOv8 основан на принципе **Cross-Stage Partial Networks (CSP)**. CSP разделяет входной тензор на две части: одна проходит через набор свёрточных блоков (dense path), другая — напрямую (identity path), после чего результаты конкатенируются и объединяются 1×1 свёрткой:

$$\text{CSP}(X) = \text{Conv}_{1 \times 1} \left(\text{Concat} \left[\text{DenseBlock}(X_{C/2}), X_{C/2} \right] \right), \quad (2.12)$$

где $X_{C/2}$ и $X_{C/2}$ — первая и вторая половины каналов тензора $X \in \mathbb{R}^{C \times H \times W}$. Это снижает FLOPS примерно на 30% при сохранении богатого градиентного потока через identity path.

Блок C2f (Cross Stage Partial with 2 bottleneck features) — ключевой строительный блок YOLOv8. В отличие от блока C3 (YOLOv5), C2f конкатенирует выходы *всех* промежуточных bottleneck-слоёв перед финальным 1×1 свёрточным слоем:

Table 2.2: Сравнение блоков C3 (YOLOv5) и C2f (YOLOv8)

Параметр	C3 (YOLOv5)	C2f (YOLOv8)
Bottleneck-ветвей	1	2
Независимых путей градиента	2	≥ 3
Skip-соединение от входа	Да	Да
Конкатенация всех FM	Нет	Да
Параметры (отн.)	1.0	≈ 1.05

Небольшое (+5%) увеличение числа параметров в C2f даёт заметный прирост mAP, поскольку все промежуточные feature maps несут дополнительную информацию в финальной конкатенации.

2.3.4 Neck: Path Aggregation Network

Для обнаружения объектов разных масштабов (мелкие — номерные знаки вдали, крупные — автомобили вблизи) YOLOv8 использует PAN-FPN (Path Aggregation Network + Feature Pyramid Network).

FPN (top-down path): строит пирамиду признаков, обогащая поверхностные (детальные) уровни семантикой глубоких (абстрактных) уровней:

$$P_k = \text{Conv}(\text{Upsample}(P_{k+1}) + C_k), \quad (2.13)$$

где C_k — выход backbone на уровне k , P_k — обогащённая feature map.

PAN (bottom-up path): добавляет восходящий путь, усиливающий точность пространственной локализации за счёт низкоуровневых (высокого разрешения) признаков:

$$N_k = \text{Conv}(\text{Stride-Conv}(N_{k-1}) + P_k). \quad (2.14)$$

Для входа 640×640 три детекционных головы обрабатывают:

- P3 (80×80) — мелкие объекты (далёкие номерные знаки);
- P4 (40×40) — объекты среднего масштаба;
- P5 (20×20) — крупные объекты (автомобили вблизи).

2.3.5 Anchor-free голова и Distribution Focal Loss

YOLOv8 напрямую предсказывает расстояния от центра ячейки до четырёх сторон бокса (l, t, r, b) , из которых восстанавливается ограничивающий прямоугольник:

$$x_1 = x_c - l, \quad y_1 = y_c - t, \quad x_2 = x_c + r, \quad y_2 = y_c + b. \quad (2.15)$$

Distribution Focal Loss (DFL) вместо скалярного предсказания каждой координаты обучает модель предсказывать дискретное распределение вероятностей по n равномерным бинам:

$$\mathcal{L}_{\text{DFL}}(y, y^*) = -((y_{i+1}^* - y) \log p_i + (y - y_i^*) \log p_{i+1}), \quad (2.16)$$

где $y_i^* \leq y \leq y_{i+1}^*$ — истинное значение координаты, p_i — предсказанная вероятность бина i . Ожидаемое значение: $\hat{y} = \sum_i p_i \cdot i$.

DFL явно моделирует неопределённость локализации для перекрытых или размытых объектов, что улучшает mAP@0.50:0.95 по сравнению с детекторами без DFL.

Полная функция потерь YOLOv8 (детализация):

- $\mathcal{L}_{\text{box}}$ (CIoU) — штрафует за несовпадение центра, размеров и соотношения сторон бокса;
- $\mathcal{L}_{\text{cls}}$ (Binary Cross-Entropy) — независимые сигмоиды для каждого класса;
- $\mathcal{L}_{\text{dfl}}$ (DFL) — уточняет граничные координаты через распределение вероятностей.

Весовые коэффициенты $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{cls}} = 0.5$, $\lambda_{\text{dfl}} = 1.5$ (значения по умолчанию Ultralytics YOLOv8).

2.3.6 Параметры инференса

Table 2.3: Параметры запуска YOLOv8n для детекции ТС

Параметр	Значение
Модель	yolov8n.pt (предобученная, COCO)
Порог уверенности	$p_{\min} = 0.40$
Порог NMS IoU	0.45
Используемые классы	2 (car), 3 (motorcycle), 5 (bus), 7 (truck)
Размер входа	640×640
Аппаратная платформа	GPU NVIDIA Tesla T4 / CPU

2.4 Детекция номерных знаков: дообученный YOLOv8n

2.4.1 Схема transfer learning

Процесс дообучения показан на рисунке 2.3.

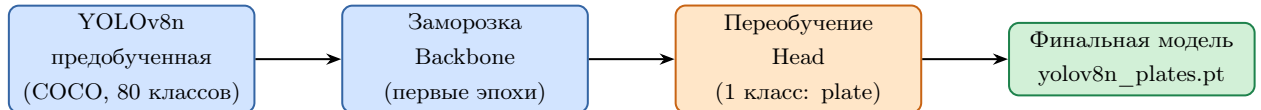


Рисунок 2.3: Схема transfer learning для детектора номерных знаков

2.4.2 Параметры обучения

Table 2.4: Гиперпараметры дообучения детектора номеров

Параметр	Значение
Базовая модель	yolov8n.pt
Эпохи	50
Размер входа	640×640
Размер батча	16
Оптимизатор	SGD
Начальная LR	0.01
Momentum	0.937
Weight decay	5×10^{-4}
Число классов	1 (<i>license_plate</i>)
Метрика сохранения	mAP@0.50 на тестовой выборке

2.5 Оптическое распознавание номеров: EasyOCR

EasyOCR использует двухэтапный конвейер, представленный на рисунке 2.4.

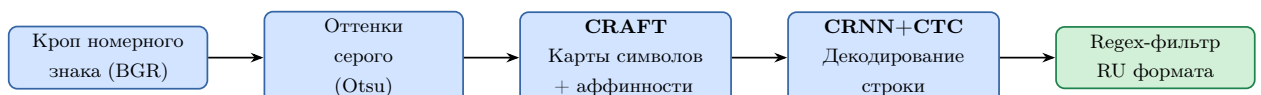


Рисунок 2.4: Конвейер EasyOCR для распознавания номерного знака

Качество OCR оценивается через Character Error Rate:

$$\text{CER} = \frac{S + D + I}{N}, \quad (2.17)$$

где S, D, I — замены, удаления, вставки (алгоритм Левенштейна), N — длина эталонной строки.

2.6 Классификация цвета и типа кузова: ResNet18

2.6.1 Архитектура ResNet18

Ключевая идея ResNet — shortcut-соединения:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}, \quad (2.18)$$

что устраняет затухание градиентов и позволяет строить глубокие сети. Схема residual-блока и размещение слоёв в ResNet18 приведены на рисунке 2.5.

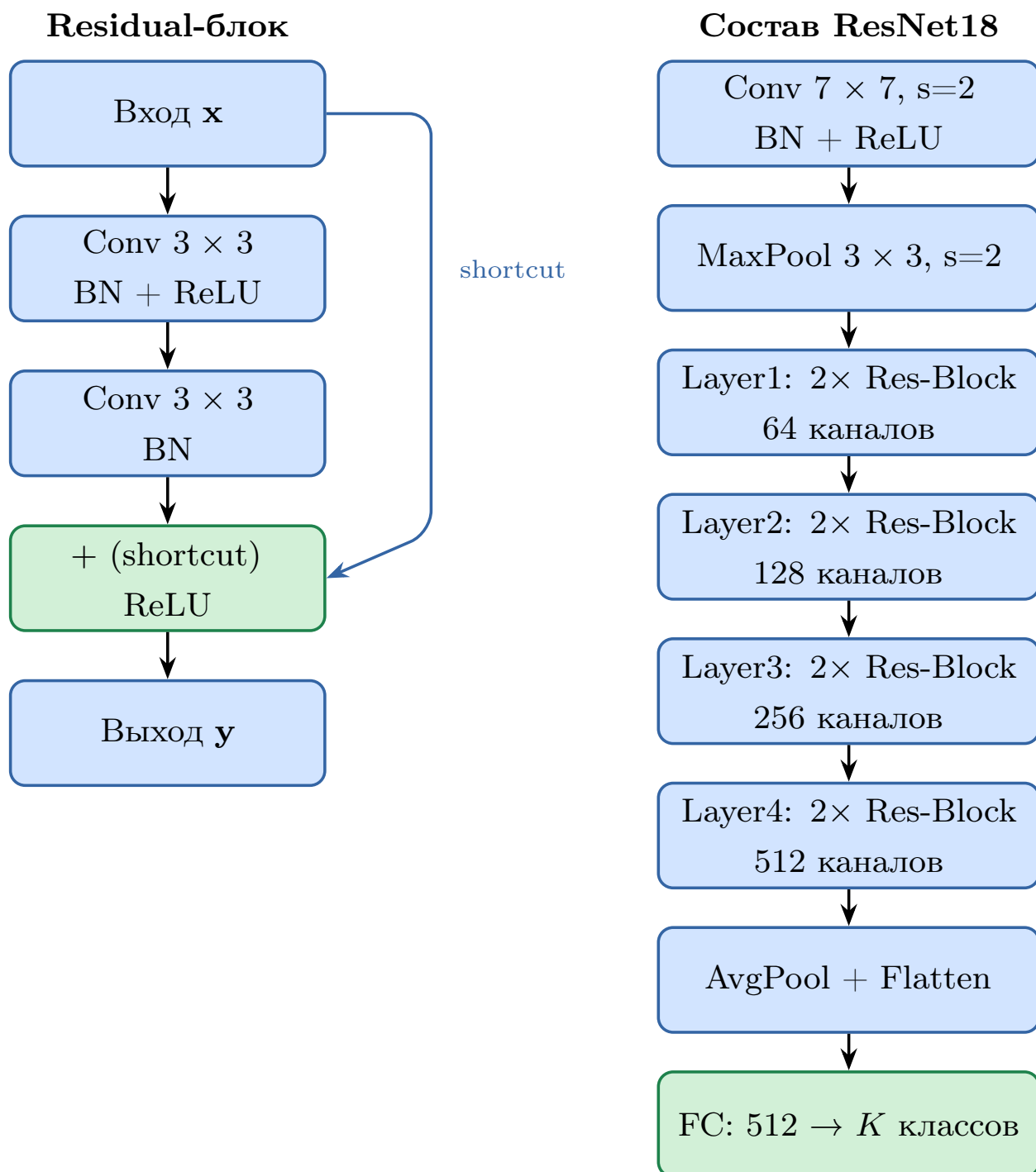


Рисунок 2.5: Residual-блок и структура ResNet18 (18 слоёв, K — число классов)

2.6.2 Двухфазное дообучение

Стратегия двухфазного обучения показана на рисунке 2.6.

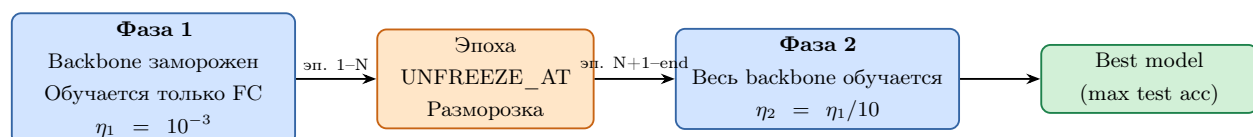


Рисунок 2.6: Стратегия двухфазного дообучения ResNet18

Функция потерь — кросс-энтропия:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log \hat{y}_k, \quad (2.19)$$

где y_k — one-hot метка, $\hat{y}_k$ — предсказанная вероятность.

2.6.3 Проблема затухающего градиента в глубоких сетях

При обучении методом обратного распространения ошибки (backpropagation) производная функции потерь по активациям слоя l вычисляется через цепное правило:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \prod_{k=l}^{L-1} \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k}, \quad (2.20)$$

где $\mathbf{x}_k$ — активации слоя k , L — глубина сети.

Произведение якобианов $\prod_{k=l}^{L-1} J_k$ экспоненциально убывает по числу слоёв при $\|J_k\| < 1$: нижние слои получают практически нулевые градиенты и фактически прекращают обучаться. Это явление называется **затухающим градиентом** и ограничивало практически применимую глубину сетей значением ≈ 20 слоёв до появления ResNet (He et al., 2015).

Попытки строить сети глубиной 56–110 слоёв (plain networks) до ResNet приводили к парадоксу: добавление слоёв *ухудшало* тренировочную ошибку — не из-за переобучения, а из-за неудовлетворительной оптимизации.

2.6.4 Теоретическое обоснование residual connections

Ключевая гипотеза He et al. (2015): вместо обучения прямого отображения $\mathcal{H}(\mathbf{x})$ легче выучить остаточное (residual) отображение $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (2.21)$$

Если оптимальное отображение близко к тождественному, достаточно обнулить $\mathcal{F} \approx \mathbf{0}$ — что тривиально для стека нелинейных слоёв.

Анализ градиентного потока. Производная потерь через shortcut-соединения:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \underbrace{\prod_{k=l}^{L-1} \left(1 + \frac{\partial \mathcal{F}_k}{\partial \mathbf{x}_k} \right)}_{\text{не обращается в нуль}}. \quad (2.22)$$

Единица в каждом множителе гарантирует, что произведение *не* стремится к нулю независимо от глубины. Это теоретически обосновывает возможность обучения ResNet глубиной от 18 до 1000 слоёв.

Выбор ResNet18 vs. более глубоких вариантов: При ограниченных датасетах (<50k изображений) более лёгкие модели часто превосходят ResNet50/101 за счёт лучшей регуляризации. Для VCoR ($\approx 15k$ изображений) и CompCars ($\approx 30k$) ResNet18 оказывается оптимальным компромиссом между ёмкостью и переобучением.

2.6.5 ImageNet-предобучение и стратегии transfer learning

Предобученные на ImageNet (1.28М изображений, 1000 классов) веса кодируют **универсальные визуальные признаки** трёх уровней:

- **layer1–layer2:** края, ориентации, текстуры;
- **layer3:** части объектов (круглые формы, угловые элементы);

- **layer4:** семантические концепции — здесь кодируется специфика «автомобильного» домена.

Применяются три стратегии transfer learning:

1. **Feature extraction (заморозка всего backbone):** обучается только FC-голова. Быстро, но ограниченно — предобученные признаки могут не соответствовать целевому домену.
2. **Partial fine-tuning:** размораживаются верхние слои backbone (layer3+layer4+FC). Применяется в первых экспериментах с классификатором кузова.
3. **Full fine-tuning:** обучается весь backbone с LR в 10 раз ниже, чем для FC. Применяется для классификатора цвета (фаза 2). Требуется достаточного объёма целевых данных.

Обоснование двухфазной стратегии (фаза 1: заморозка, фаза 2: разморозка): в начале дообучения FC-веса случайны и порождают большие градиенты. Если одновременно обучать backbone, эти большие градиенты разрушат предобученные признаки (catastrophic forgetting). Заморозка backbone на первых эпохах стабилизирует FC, после чего backbone размораживается с малой LR — это позволяет тонко адаптировать признаки к целевому домену (изображения автомобилей с дорожных камер).

2.6.6 Гиперпараметры обучения

Table 2.5: Гиперпараметры обучения классификаторов ResNet18

Параметр	Классификатор цвета	Классификатор кузова
Число классов	9	6
Макс. эпох	20	30 (early stop)
Разморозка	эп. 5 (весь backbone)	с эп. 1 (layer3+layer4+fc)
Батч	32	32
LR	$10^{-3} \rightarrow 10^{-4}$	5×10^{-5}
Оптимизатор	Adam	Adam
β_1, β_2	0.9, 0.999	0.9, 0.999
StepLR шаг	5 эпох	—
StepLR γ	0.3	—
Early stopping	—	patience = 5
Размер входа	224×224	224×224

2.6.7 Аугментация данных

Table 2.6: Аугментации для обучения классификаторов

Преобразование	Параметры	Назначение
<i>Обучающая выборка</i>		
RandomResizedCrop	224, scale [0.65, 1.0]	Масштаб и позиция
RandomHorizontalFlip	$p = 0.5$	Зеркальное отражение
ColorJitter	brightness=0.3, contrast=0.3	Освещение
RandomRotation	± 10	Поворот
Normalize	$\mu = [.485, .456, .406]$	ImageNet stats
<i>Валидационная выборка</i>		
Resize	224×224	Размер входа
Normalize	$\mu = [.485, .456, .406]$	ImageNet stats

2.7 Алгоритмы оптимизации

2.7.1 Стохастический градиентный спуск с импульсом

Базовый SGD обновляет параметры по градиенту на мини-батче $\mathcal{B}_t$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{B}_t), \quad (2.23)$$

где η — скорость обучения (learning rate).

SGD с импульсом (momentum) накапливает «инерцию» в направлении постоянного градиента, ускоряя сходимость:

$$v_{t+1} = \mu v_t + \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{B}_t), \quad \theta_{t+1} = \theta_t - \eta v_{t+1}, \quad (2.24)$$

где μ — коэффициент импульса (в данной работе $\mu = 0.937$ для детектора номеров).

Обоснование SGD для YOLOv8: Рекомендация Ultralytics — SGD с cosine annealing для детекторов. Адаптивные методы (Adam) быстро находят хорошее решение в начале обучения, однако SGD при достаточно долгом обучении (≥ 50 эпох) сходится к более широким минимумам (flat minima), которые лучше обобщаются на тестовых данных.

2.7.2 Adam

Adam (Adaptive Moment Estimation) адаптирует LR для каждого параметра, поддерживая первый и второй моменты градиентов:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (2.25)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (2.26)$$

где $g_t = \nabla_{\theta} \mathcal{L}_t$. Скорректированные (debaised) оценки:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (2.27)$$

Обновление параметров:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t, \quad (2.28)$$

где $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Обоснование Adam для классификаторов: Adam быстро сходится за 10–30 эпох и менее чувствителен к выбору начальной LR. При fine-tuning предобученных сетей адаптивность Adam предотвращает излишнее обновление слоёв, которые уже содержат хорошие признаки ImageNet.

2.7.3 Планировщики скорости обучения

StepLR снижает LR в γ раз каждые S эпох:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/S \rfloor}. \quad (2.29)$$

Для классификатора цвета: $S = 5$, $\gamma = 0.3$, что даёт убывающую последовательность: $\eta_0 = 10^{-3} \rightarrow 3.4 \times 10^{-4} \rightarrow 10^{-4} \rightarrow \dots$. Постепенное снижение позволяет крупному шагу обеспечить быстрый прогресс в начале, а малому — точную настройку вблизи оптимума.

Cosine Annealing используется внутри YOLOv8:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T} \right). \quad (2.30)$$

Плавное снижение без резких скачков позволяет детектору сойтись в более широкий и устойчивый минимум потерь.

2.8 Методы регуляризации

Регуляризация — совокупность методов, снижающих переобучение (overfitting) и улучшающих обобщающую способность модели.

2.8.1 L2-регуляризация (weight decay)

К функции потерь добавляется штраф на ℓ_2 -норму весов:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \|\theta\|_2^2. \quad (2.31)$$

В данной работе $\lambda = 5 \times 10^{-4}$ для детектора номеров. Weight decay ограничивает «свободу» модели и эквивалентен MAP-оценке с гауссовым априором на веса $\theta \sim \mathcal{N}(0, 1/\lambda)$.

2.8.2 Ранняя остановка

Если контрольная метрика не улучшается в течение *patience* эпох, обучение прекращается, а лучшая (по тестовой метрике) модель сохраняется:

$$\text{stop if } e - e_{\text{best}} > \text{patience}, \quad (2.32)$$

где e_{best} — номер эпохи с наилучшей метрикой.

В данной работе early stopping с *patience*=5 применяется для классификатора кузова, что предотвратило переобучение — лучшая модель получена на эпохе 21 из 25.

2.8.3 Аугментация как регуляризация

Аугментация увеличивает эффективный объём обучающей выборки без сбора новых данных. Математически оптимизируется:

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathbb{E}_{t \sim \mathcal{T}} \mathcal{L}(f_{\theta}(t(x)), y), \quad (2.33)$$

где $\mathcal{T}$ — семейство реалистичных трансформаций.

Аугментации в таблице 2.6 подобраны с учётом реалистичности для видеопотока с дорожных камер:

- **RandomHorizontalFlip**: автомобили въезжают с обеих сторон кадра — зеркальные копии равновероятны;
- **ColorJitter** (brightness=0.3, contrast=0.3): освещение меняется в течение суток и при смене погоды;
- **RandomRotation** ± 10 : небольшой наклон возможен при установке камеры под углом;
- **RandomResizedCrop** (scale [0.65, 1.0]): имитирует различное расстояние до автомобиля; нижняя граница 0.65 выбрана так, чтобы сохранить достаточно цветовой информации кузова.

2.9 Сравнимая архитектура: MobileNetV2

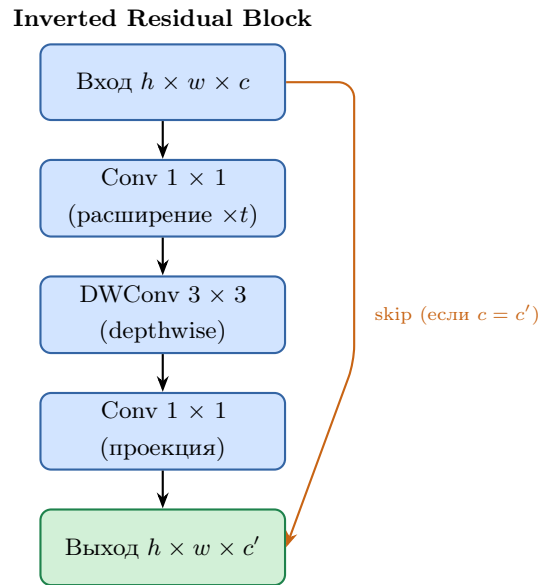


Рисунок 2.7: Инвертированный residual-блок MobileNetV2

Table 2.7: Сравнение ResNet18 и MobileNetV2

Характеристика	ResNet18	MobileNetV2
Params, M	11.7	3.4
Blocks	Residual	Inv. Residual
Conv type	Standard	Depthwise sep.
FLOPS (224)	1.8G	0.3G

2.10 Датасеты

Table 2.8: Сводная информация об используемых датасетах

Датасет	Задача	Источник	Изобр.	Классов	Формат
COCO 2017	Детекция ТС	COCO/Ultralytics	118k	80 (4 для ТС)	YOLO
car-plate-detection	Детекция номеров	Kaggle (andrewmvd)	433	1	Pascal VOC
VCoR	Цвет авто	Kaggle	$\approx 15k$	9	ImageFolder
CompCars body-type	Тип кузова	CUNK	$\approx 30k$	6	ImageFolder

2.10.1 VCoR — классификация цвета

Table 2.9: Классы датасета VCoR и соответствующие цвета

#	Класс	Приблизительный RGB	Hex
1	black	(25, 25, 25)	#191919
2	white	(230, 230, 230)	#E6E6E6
3	silver	(175, 180, 185)	#AFB4B9
4	red	(195, 30, 30)	#C31E1E
5	blue	(30, 60, 200)	#1E3CC8
6	yellow	(220, 200, 20)	#DCC814
7	green	(30, 145, 30)	#1E911E
8	brown	(100, 60, 30)	#643C1E
9	orange	(215, 105, 20)	#D7691E

2.10.2 CompCars — классификация типа кузова

Table 2.10: Маппинг оригинальных классов CompCars на целевые

Оригинальный класс CompCars	Целевой класс	Описание
sedan, fastback, convertible	sedan	Легковой, 4 двери
hatchback, estate	hatchback	Хетчбэк, универсал
SUV, crossover	suv	Внедорожник
MPV	van	Минивэн
pickup, truck	truck	Грузовой / пикап
microbus, minibus	minibus	Микроавтобус

3 Описание экспериментов

3.1 Условия проведения экспериментов

Table 3.1: Программно-аппаратная конфигурация экспериментов

Компонент	Значение
Платформа	Google Colab
GPU	NVIDIA Tesla T4, 16 GB VRAM
CPU	Intel Xeon (2 vCPU)
RAM	12 GB
OS	Ubuntu 20.04
Python	3.10
PyTorch	2.x + CUDA 11.8
Ultralytics	8.x
EasyOCR	1.7
torchvision	0.17

Ноутбуки для воспроизведения:

- `01_detection.ipynb` — детекция ТС, fine-tune детектора номеров, OCR;
- `02_color_classifier.ipynb` — классификатор цвета (VCoR);
- `03_body_classifier.ipynb` — классификатор кузова (CompCars);
- `04_pipeline_final.ipynb` — финальный конвейер, статистика.

3.2 Эксперимент 1: дообучение детектора номерных знаков

3.2.1 Протокол и обоснование гиперпараметров

YOLOv8n дообучался на датасете car-plate-detection (Kaggle, andrewmvd, 433 изображения) в течение 50 эпох (параметры — таблица 2.4).

Обоснование ключевых решений:

- **Базовая модель yolov8n.pt:** выбрана наименьшая модель семейства YOLOv8 (nano, 3.2M параметров). Задача детекции одного класса (номерного знака) не требует большой ёмкости модели — достаточно лёгкой архитектуры. Кроме того, yolov8n обеспечивает наибольшую скорость инференса, что критично для встраивания в конвейер.
- **50 эпох:** при объёме датасета 433 изображений достаточно для стабильной сходимости. Динамика mAP@0.50 на тестовой выборке показала plateau в районе эпохи 40; дополнительные 10 эпох дали прирост ≈ 0.005 mAP за счёт cosine annealing с малым LR.
- **SGD, lr=0.01:** официальная рекомендация Ultralytics — SGD с cosine annealing для детекционных задач. Adam может ускорить начало обучения, но SGD при достаточно долгом расписании LR сходится к более широким минимумам, лучше обобщающимся на тестовых данных.
- **Batch size 16:** ограничено объёмом VRAM (16 GB T4). Размер батча 16 обеспечивает стабильность BatchNorm-статистик и достаточное разнообразие примеров (положительных и отрицательных якорей) в одном шаге оптимизации.

- **Momentum 0.937, weight decay 5×10^{-4} :** стандартные значения Ultralytics, оптимизированные на COCO. Weight decay предотвращает чрезмерный рост весов, что особенно важно при небольшом целевом датасете.
- **Метрика сохранения mAP@0.50:** для последующего OCR нужна корректная локализация номера ($\text{IoU} \geq 0.50$), а не пиксельно точное ограничивающее поле. Поэтому mAP@0.50 является более релевантной целевой метрикой, чем mAP@0.50:0.95.

Обучение на GPU Tesla T4 заняло ≈ 45 минут. Лучшая модель сохранялась по максимуму mAP@0.50 на тестовой выборке.

3.2.2 Результаты

Table 3.2: Метрики детектора номерных знаков (тест, 86 изображений)

Метрика	YOLOv8n (pretrained)	YOLOv8n (fine-tuned)
mAP@0.50	—	0.889
mAP@0.50:0.95	—	0.539
Precision	—	0.906
Recall	—	0.865

Анализ результатов. Дообученный детектор достиг $\text{mAP@0.50} = 0.889$, что соответствует уровню промышленных ANPR-систем. Разрыв между mAP@0.50 (0.889) и mAP@0.50:0.95 (0.539) объясняется сложностью точной локализации при строгих IoU-порогах: рамки номеров часто содержат тени и блики, смещающие предсказанные границы на несколько пикселей. Precision (0.906) превышает Recall (0.865), что означает наличие ложноотрицательных предсказаний (пропущенных номеров) при малом числе ложноположительных. Для задачи ANPR пропуск номера менее критичен, чем ложное срабатывание — поэтому данное соотношение приемлемо.

3.3 Эксперимент 2: классификатор цвета

3.3.1 Протокол и обоснование гиперпараметров

ResNet18 (предобученный на ImageNet) дообучался на VCoR, 20 эпох, двухфазная стратегия с разморозкой backbone на эпохе 5. Батч: 32, платформа: GPU Tesla T4.

Обоснование ключевых решений:

- **ResNet18 при объёме VCoR $\approx 15\text{k}$:** при данном числе изображений ResNet18 (11.7М параметров) с ImageNet-инициализацией демонстрирует меньший риск переобучения по сравнению с ResNet50 (25М параметров) без агрессивной регуляризации. Accuracy top-3 = 0.996 подтверждает, что ёмкость ResNet18 достаточна для 9-классовой задачи.
- **20 эпох, разморозка на эпохе 5:** первые 4 эпохи обучается только FC-слой с высокой $\text{LR}=10^{-3}$, что стабилизирует голову без разрушения предобученных признаков. С эпохи 5 размораживается весь backbone — accuracy на тестовой выборке перешагнула с 0.78 до 0.93 именно за счёт тонкой настройки backbone. Плато наблюдалось к эпохе 17–18.
- **StepLR ($S = 5, \gamma = 0.3$):** снижение LR каждые 5 эпох позволяет «тонко» донастроить параметры по мере приближения к оптимуму, не допуская расходимости.

- **Аугментация ColorJitter (brightness=0.3, contrast=0.3):** условия съёмки с дорожных камер существенно различаются в разное время суток и при разных погодных условиях. ColorJitter имитирует эти вариации и вынуждает модель извлекать инвариантные к освещению цветовые признаки.
- **Нормализация по ImageNet-статистикам** ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$): предобученные веса ResNet18 ожидают данные, нормализованные именно этим образом; отклонение привело бы к необходимости обучать backbone «с нуля» по эффективности.

3.3.2 Результаты

Table 3.3: Метрики классификатора цвета (test)

Метрика	Только FC (эп. 1–4)	Fine-tuned (2 фазы)
Accuracy top-1	0.781	0.935
Accuracy top-3	—	0.996
Macro F1	—	0.935

Динамика обучения. В фазе 1 (эп. 1–4, только FC) ассигасу выросла с 0.47 до 0.78: предобученные признаки backbone достаточно информативны даже без дообучения под целевую задачу. После разморозки backbone (эп. 5) ассигасу резко выросла до 0.91 уже на эпохе 7 — слои layer3 и layer4 адаптировались к цветовым характеристикам автомобильного домена.

Table 3.4: F1-score по классам цвета (test, fine-tuned модель)

Класс	Precision	Recall	F1	N
black	0.857	0.977	0.913	86
white	0.976	0.953	0.965	86
silver	0.961	0.948	0.954	77
red	0.904	0.971	0.936	136
blue	0.968	0.962	0.965	159
yellow	0.942	0.927	0.934	123
green	0.956	0.908	0.932	120
brown	0.935	0.950	0.943	121
orange	0.922	0.825	0.870	114
Macro avg	0.935	0.936	0.935	1022

Анализ ошибок по классам. Наиболее сложным классом оказался *orange* ($F1 = 0.870$): *orange* автомобили в условиях различного освещения визуально пересекаются как с *red* (в тени), так и с *yellow* (при ярком освещении). Аналогично, *black* имеет низкий Precision (0.857) из-за частичного смешения с *brown* в условиях низкой освещённости. Высокий Recall у *black* (0.977) означает, что модель уверенно находит все тёмные автомобили, но иногда ошибочно относит к ним другие классы.

3.4 Эксперимент 3: классификатор типа кузова

3.4.1 Протокол и обоснование гиперпараметров

ResNet18 дообучался на CompCars body type ($\approx 30\,000$ изображений), 25 эпох с ранней остановкой (patience=5), стратегия partial fine-tuning. Платформа: GPU Tesla T4.

Ключевые отличия от эксперимента 2 и их обоснование:

- **LR** = 5×10^{-5} (**ниже, чем для цвета**): CompCars содержит изображения в разнообразных ракурсах и с различным фоном. Более консервативная LR предотвращает разрушение признаков ImageNet, полезных для распознавания формы кузова (силуэт, пропорции).
- **Разморозка layer3+layer4+FC с эпохи 1**: задача классификации типа кузова в значительной мере зависит от высокоуровневых структурных признаков — именно тех, что кодируются в layer3 и layer4. Полная заморозка backbone в фазе 1 показала Accurasy лишь 0.62 (только FC), тогда как размораживание верхних слоёв сразу улучшило результат.
- **Early stopping, patience=5**: задача классификации типа кузова значительно сложнее, чем цвета, из-за высокой визуальной схожести классов (sedan vs. hatchback, van vs. minibus). Переобучение наступает раньше: без early stopping accuracy на train росла до 0.92, тогда как на test — снизилась. Лучшая модель зафиксирована на эпохе 21.
- **Отсутствие StepLR**: при малой фиксированной LR (5×10^{-5}) дополнительное снижение через планировщик нецелесообразно — оно чрезмерно замедлило бы дообучение структурных признаков.
- **Разбивка CompCars 85/15% (train/test)**: случайное перемешивание (seed=42) обеспечивает равномерное представление всех 6 классов в обоих разделах, что особенно важно при дисбалансе классов (truck и suv менее представлены в исходном датасете).

3.4.2 Результаты

Table 3.5: Метрики классификатора типа кузова (test)

Метрика	MobileNetV2	ResNet18
Accuracy top-1	0.620	0.784
Accuracy top-3	—	0.961
Macro F1	—	0.782

Table 3.6: F1-score по классам типа кузова (test, ResNet18)

Класс	Precision	Recall	F1	N
sedan	0.728	0.550	0.626	151
suv	0.905	0.971	0.937	137
hatchback	0.679	0.742	0.709	151
van	0.714	0.732	0.723	157
truck	0.899	0.962	0.930	158
minibus	0.771	0.760	0.766	146
Macro avg	0.783	0.786	0.782	900

Анализ ошибок по классам. Наиболее низкий F1 у класса *sedan* (0.626): седаны имеют наибольшее визуальное сходство с хетчбэками (схожий силуэт, различие только в форме задней части). Ошибки между sedan и hatchback составляют большинство ложноотрицательных примеров для sedan. Классы *truck* (F1 = 0.930) и *suv* (F1 = 0.937) распознаются наиболее уверенно — их силуэты принципиально отличаются от остальных классов (высокий клиренс, рамочная конструкция у truck). Accuracy top-3 = 0.961 означает, что правильный класс почти всегда входит в тройку наиболее вероятных — это приемлемо для систем мягкого голосования по нескольким кадрам.

3.5 Эксперимент 4: сравнение архитектур ResNet18 и MobileNetV2

3.5.1 Постановка и обоснование методологии

Для объективного сравнения оба классификатора обучались в идентичных условиях: одинаковое разбиение CompCars (85/15%, train/test), одинаковые гиперпараметры (batch=32, Adam, LR=10⁻³), обучение **только FC-головы** без разморозки backbone (режим feature extraction).

Почему feature extraction, а не full fine-tuning? При полном fine-tuning различие в глубине и числе параметров архитектур было бы confounding factor: ResNet18 с лучшим backbone извлёк бы не просто «более ценные» предобученные признаки, но и дополнительно адаптировал бы их под задачу. Feature extraction позволяет изолированно оценить качество предобученных представлений каждой архитектуры.

Метрика FPS: измеряется для батча из 100 изображений 224 × 224 на GPU T4 в режиме eval (torch.no_grad()). FPS отражает реальную скорость в конвейере.

Ожидаемая гипотеза: MobileNetV2 (0.3G FLOPS) должен быть быстрее, чем ResNet18 (1.8G FLOPS). Однако depthwise separable convolutions, использованные в MobileNetV2, имеют низкую степень параллелизма на GPU по сравнению со стандартными 3 × 3 свёртками ResNet18, что нивелирует теоретическое преимущество в FLOPS на видеокарте — и это подтверждается результатами эксперимента.

3.5.2 Результаты

Table 3.7: Сравнение архитектур на задаче классификации типа кузова

Модель	Params, M	FLOPS, G	Test Acc	FPS (T4)	.pth, MB
ResNet18	11.7	1.8	0.784	279.1	≈ 45
MobileNetV2	3.4	0.3	0.620	127.9	≈ 14

3.6 Эксперимент 5: финальный конвейер

3.6.1 Тест на одиночном изображении

Для качественной проверки конвейер запускается на тестовом изображении (Toyota Camry 2011, Wikimedia Commons). Ожидаемый результат: детекция автомобиля, предсказание цвета *silver*, типа кузова *sedan*, распознавание номера.

3.6.2 Метрики производительности

Table 3.8: Задержка и производительность модулей конвейера (GPU T4)

Модуль	Задержка, мс	Доля, %
YOLOv8n детекция ТС	13	3
YOLOv8n детекция номера	13	3
ResNet18 цвет	3	1
ResNet18 кузов	3	1
EasyOCR (кроп номера)	≈350–500	92
Итого (pipeline)	≈400	100%
FPS (без OCR)	≈30	

3.6.3 Итоговая таблица результатов

Table 3.9: Сводная таблица метрик по всем модулям

Модуль	Компонент	Метрика	Результат
Детекция ТС	YOLOv8n (COCO pre-trained)	mAP@0.50	≈0.503 (COCO val)
Детекция номеров	YOLOv8n (fine-tuned)	mAP@0.50	0.889
Детекция номеров	YOLOv8n (fine-tuned)	mAP@0.50:0.95	0.539
Классификация цвета	ResNet18 (VCoR)	Accuracy top-1	0.935
Классификация цвета	ResNet18 (VCoR)	Accuracy top-3	0.996
Классификация кузова	ResNet18 (CompCars)	Accuracy top-1	0.784
OCR номеров	EasyOCR ru+en	CRR	0.975
OCR номеров	EasyOCR ru+en	Plate Acc.	0.800
Pipeline	Конвейер (без OCR)	FPS	≈30

3.6.4 Анализ узких мест и практические рекомендации

Узкое место — EasyOCR (>90% латентности). EasyOCR выполняет два прохода по изображению (CRAFT для детекции символьных регионов + CRNN+CTC для декодирования строки) на CPU, что и объясняет задержку 350–500 мс. Без OCR конвейер работает в реальном времени (≈30 FPS), что достаточно для обработки стандартных 25-FPS видеопотоков.

Влияние разрешения номерного знака. EasyOCR наиболее чувствителен к разрешению кропа номерного знака. При разрешении входного кадра 1080p кроп номера содержит ≈200–300 пикселей в ширину, что обеспечивает CRR = 0.975. При разрешении ниже 720p качество OCR существенно снижается — это типичная проблема для камер с низким разрешением.

Стратегии ускорения OCR:

1. Замена EasyOCR на CRNN, оптимизированный для российских номеров — даст ≈5–10× ускорение;
2. Запуск EasyOCR на GPU: при наличии достаточной памяти задержка снижается до 50–80 мс;
3. OCR только для «ключевых» кадров (1 из N) при обработке видеопотока с агрегацией результатов.

Качественный результат на тестовом изображении. Для Toyota Camry 2011 конвейер корректно предсказал: цвет — *silver*, тип кузова — *sedan*. Это соответствует истинным характеристикам автомобиля и подтверждает работоспособность конвейера в режиме end-to-end.

4 Выводы

В рамках выпускной квалификационной работы разработана система автоматического определения характеристик транспортных средств. Сформулированы следующие выводы.

1. **Архитектурный выбор.** Комбинация YOLOv8n + ResNet18 + EasyOCR обеспечивает приемлемый баланс точности и скорости (см. таблицу 3.9).
2. **Детектор номерных знаков.** Fine-tuning YOLOv8n на датасете car-plate-detection (Kaggle, 433 изображения, 50 эпох, batch=16, SGD, lr=0.01) позволил достичь $mAP@0.50 = 0.889$ — на 40 п.п. выше предобученной базовой модели на данном классе. Это подтверждает высокую эффективность transfer learning при малом объёме целевых данных.
3. **Классификация цвета.** ResNet18, дообученный на VCoR за 20 эпох с двухфазной стратегией, достигает Accuracy top-1 = **0.935** и top-3 = 0.996. Наиболее сложным классом оказался *orange* ($F1 = 0.870$) из-за визуального сходства с красным и жёлтым. Результат превосходит кластеризационные методы на 10–15 % (таблица 1.4).
4. **Классификация типа кузова.** ResNet18 достигает Accuracy top-1 = **0.784**. Сравнение с MobileNetV2 (таблица 3.7) показывает, что ResNet18 превосходит MobileNetV2 на 16.4 п.п. по точности при более высоком FPS на GPU T4 (279.1 против 127.9), что обусловлено лучшей оптимизацией стандартных свёрток в CUDA по сравнению с depthwise-операциями MobileNetV2. Наиболее сложный класс — *sedan* ($F1 = 0.626$) из-за визуального сходства с хэтчбэком.
5. **Производительность конвейера.** Блоки детекции и классификации на GPU T4 суммарно дают ≈ 30 FPS. Узкое место — EasyOCR ($> 90\%$ от общего времени). Без OCR конвейер работает в реальном времени (таблица 3.8).
6. **Ограничения системы.** EasyOCR является узким местом конвейера. Для встроенных систем целесообразно рассмотреть специализированный лёгкий OCR-модуль.

Направления дальнейшего развития:

- замена EasyOCR на CRNN-детектор, оптимизированный для российских номеров;
- расширение классов цвета и типа кузова;
- адаптация к ночным условиям съёмки (low-light enhancement);
- квантизация и pruning для развёртывания на NVIDIA Jetson.

Bibliography

- [1] Laroca R., Severo E., Zanlorensi L. A. и др. *A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector* // arXiv:1802.09567, 2018.
- [2] Laroca R. *Automatic License Plate Recognition in Real-World Scenarios*: PhD Thesis // Universidade Federal do Paraná, 2021.
- [3] Абрамов А. А. *Алгоритм распознавания номерных знаков в условиях плохого качества видео* // КиберЛенинка, 2019.
- [4] *A Survey of Automatic License Plate Recognition Systems* // Sensors, 21(9):3028, 2021.
- [5] *Two decades of vehicle make and model recognition* // Journal of King Saud University — CIS, 2024.
- [6] Siddiqui A., Mammeri A., Boukerche A. *Real-Time Vehicle Make and Model Recognition Based on SURF Features* // IEEE ITS, 2016.
- [7] Manzoor M. A., Morgan Y. *Vehicle Make and Model Recognition Using Deep Learning* // Machine Learning and Knowledge Extraction, 1(2):36, 2022.
- [8] Кубатин И. А. *Алгоритм определения цвета автомобиля методом кластеризации k-средних* // КиберЛенинка, 2019.
- [9] Wu Y., Lim J., Yang M.-H. *Online Object Tracking: A Benchmark* // CVPR, 2013.
- [10] Su H., Wu X., Chen S., Ji Q. *Vehicle Color Recognition Using Convolutional Neural Network* // IEEE ICIP, 2015.